

Real-Time Analytics Dashboard Report

The main idea of this project is to build a real-time analytics dashboard that collects metrics from many servers, processes them, and displays updated results for engineers monitoring system performance. The system receives data from thousands of servers, each sending metrics such as CPU or memory usage. The goal is to design and test different communication and data ingestion patterns to see how each affects performance, latency, and scalability.

To achieve this, I implemented three different approaches:

- **Approach 1:** Stateless architecture using HTTP/1.1 protocol with simple HTTP POST requests.
- **Approach 2:** Stateful architecture using WebSocket communication over HTTPS.
- **Approach 3:** Stateful streaming model using WebSocket with HTTP/2 and multiplexing support.

Let's start by explaining each approach individually.

Approach 1: Stateless HTTP/1.1 with POST Requests

Overview:

This approach represents the simplest implementation of the system. It focuses on stateless communication between the servers and the backend using basic HTTP POST requests over the HTTP/1.1 protocol.

- Built a stateless backend that handles incoming metrics through **HTTP POST** requests.
- Each server sends metrics to the endpoint **/api/metrics**.
- The backend processes each request sequentially and stores raw data in **MetricRaw**.
- Aggregation logic runs immediately to calculate **count, sum, min, max, and p95** values stored in **MetricMinuteAgg**.
- No in-memory state is kept; all data is persisted in **MySQL**, allowing easy horizontal scaling.

Load Distribution:

To balance the incoming load, a lightweight proxy server was introduced.

- Implemented an **HTTP proxy (port 4000)** using **round-robin** load balancing.
- The proxy forwards requests alternately to two backend instances (**port 3001 and port 3002**).
- This setup evenly distributes concurrent traffic and slightly reduces average latency under load.

Architecture Layers:

The system is divided into clear functional layers to simplify design and scalability.

- **Data ingestion layer:** Receives metric updates through HTTP POST requests.
- **Aggregation layer:** Executes calculations using **Sequelize ORM**.
- **Storage layer:** Persists both raw and aggregated metrics in **MySQL**.
- **Proxy layer:** Handles request routing between backend instances.

Communication Characteristics:

This design relies on the HTTP/1.1 request–response model.

- Each HTTP request opens a separate connection, increasing overhead at scale.
- The dashboard retrieves updates manually from **/api/dashboard**, which works as **polling**, not real-time streaming.
- As a result, updates appear with short delays rather than instantly.

Testing Observations:

Testing was performed using **k6** to measure performance under simulated load.

- Used **100 virtual users** sending continuous metrics for **one minute**.
- The backend handled all requests successfully without errors.
- Latency was stable at moderate load but increased when concurrency grew.
- The proxy balanced requests efficiently and reduced average response time slightly.
- Synchronous aggregation caused noticeable slowdown under very high load.

When to Use This Approach:

Approach 1 works best in simpler environments or when real-time streaming is unnecessary.

- Suitable for systems that prioritize **accuracy and reliability** over instant updates.
- Easy to **deploy, debug, and scale horizontally** by adding backend instances.
- Useful when real-time requirements are low and system simplicity is a priority.

Limitations:

While stable and simple, this design has several clear drawbacks.

- Higher latency due to per-request aggregation.
- No persistent connection for live dashboard updates.
- Inefficient under very high-frequency metric ingestion.

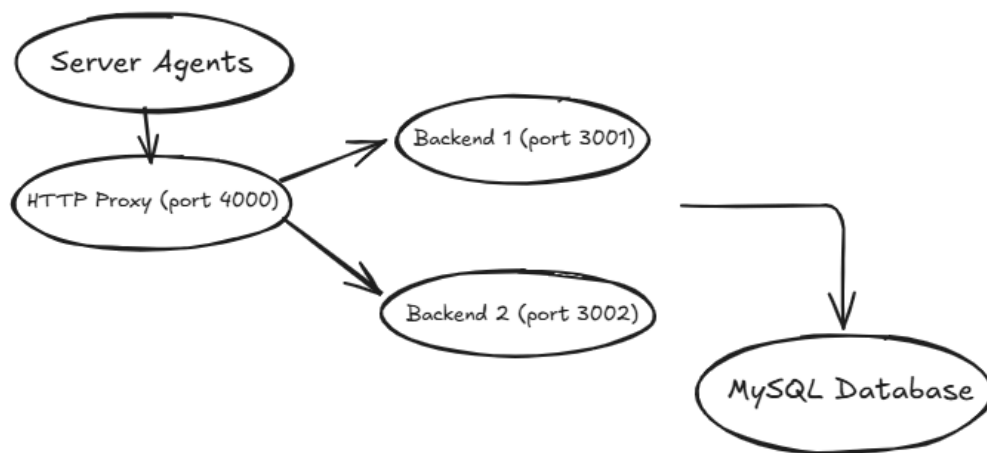
Real-World Considerations:

This design performs well for smaller or moderate-scale systems.

- **Low cost and maintenance** due to minimal dependencies.
- **Easy to scale horizontally** by adding identical stateless instances.
- Limited in supporting real-time dashboards where constant data flow is needed.

Architecture Diagram:

Represents the full system structure to visualize data flow.



The **dashboard** connects separately to the backend using **GET /api/dashboard** → **Backend** to fetch recent aggregated data.

OSI Model Breakdown:

- **Layer 7:** HTTP/1.1 REST API
- **Layer 4:** TCP
- **Layer 3:** IP
- **Layers 1–2:** Underlying network hardware

Design Decisions and Trade-offs:

- Choose **HTTP POST** for simplicity and universal compatibility.
- Adopted **stateless design** for straightforward horizontal scaling.
- Used **round-robin proxy** for balanced load distribution.
- Accepted **higher latency and connection overhead** as the trade-off for design simplicity.

Approach 2: Real-Time HTTPS + WebSockets (Socket.IO)

Overview:

This approach improves upon the first one by addressing its main limitation—lack of real-time updates.

In the first approach, the dashboard relied on repeated HTTP polling, which increased latency and resource usage.

To solve that, this design introduces **persistent HTTPS connections with WebSockets**, enabling instant metric delivery from backend to dashboard clients without constant polling.

Data Flow:

Explains how data moves through the system components.

- Server Agents → send metrics as HTTPS POST requests to `/api/metrics`.
- HTTPS Proxy (port 4444) → distributes incoming traffic between Backend 1 (port 4002) and Backend 2 (port 4003).
- Backends → store incoming metrics in MySQL and in a temporary in-memory buffer.
- Aggregation module → periodically aggregates buffered metrics and emits live results.
- Dashboard → connects to the proxy using WebSockets to receive continuous metric updates.

Architecture Description:

Summarizes how each layer functions together.

- **Proxy layer:** Implements secure HTTPS load balancing using `http-proxy` with round-robin logic.
- **Backend layer:** Express-based servers receive metrics, handle WebSocket connections, and expose API endpoints.
- **Aggregation layer:** Uses an in-memory data buffer to compute average, min, max, and p95 every second.
- **Storage layer:** MySQL stores all raw metrics for historical analysis.
- **Dashboard layer:** Frontend uses Socket.IO to visualize metrics instantly as they arrive.

Core Components:

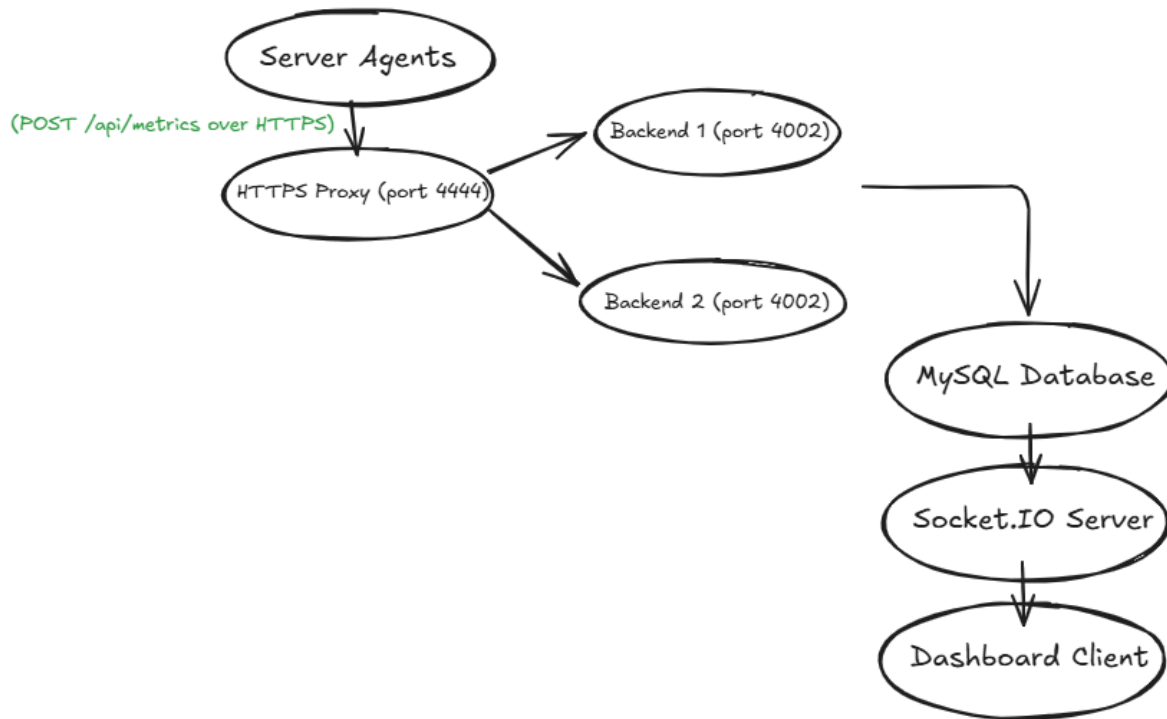
Highlights the main scripts that form this architecture.

- `proxy.js` → Handles HTTPS routing, WebSocket upgrades, and load balancing.
- `server.js` → Hosts APIs, serves the dashboard UI, and integrates Socket.IO for live communication.
- `memoryAggregator.js` → Buffers and aggregates metrics in memory, then emits real-time results.

- `metric.controller.js` → Validates and stores incoming metrics into MySQL, and triggers the aggregator.
- `dashboard.html` → Connects via WebSocket and displays live metrics updates from the proxy.

Architecture Diagram (for this approach):

Represents the full system structure to visualize data flow.



OSI Model Breakdown:

Shows which OSI layers are active in this design.

- Layer 7: HTTPS + WebSocket protocol (Socket.IO)
- Layer 4: TCP
- Layer 3: IP
- Layers 1–2: Network hardware

Real-Time Mechanism:

Explains how metrics are streamed in real time.

- Socket.IO establishes a persistent TCP connection for bi-directional communication.
- The in-memory aggregator emits metrics every second through WebSocket events.
- The dashboard receives these updates instantly, removing the need for polling.

Load Distribution:

Describes how traffic and processing are balanced.

- Proxy uses round-robin scheduling between Backend 1 and Backend 2.
- Each backend handles both REST API and WebSocket connections.
- The proxy supports `allowHTTP1: true` to ensure HTTPS fallback for non-WebSocket clients.

Performance Behavior:

Summarizes test results and observed performance.

- In-memory aggregation greatly reduces latency for dashboard updates.
- Persistent connections improve throughput compared to per-request HTTP calls.
- Under high load (100 users), latency remained below one second.
- Proxy maintained even load distribution across backend servers.

Testing Observations:

Documents how this approach performed under simulated load.

- All metrics were processed successfully under concurrent requests.
- WebSocket connections remained stable during the test duration.
- Dashboard reflected updates instantly (<1 second delay).
- MySQL write operations stayed consistent with no data loss.

When to Use This Approach:

Defines best-fit use cases.

- Ideal for systems needing near-real-time data visibility.
- Suitable for performance monitoring and IoT dashboards.
- Recommended when balancing between scalability and live interactivity.

Limitations:

Outlines the trade-offs of this design.

- Slightly higher resource usage due to persistent WebSocket connections.
- Aggregated data stored in memory is temporary and lost on restart.
- HTTPS setup requires valid SSL certificates.
- Slightly more complex proxy setup compared to HTTP-only mode.

Design Decisions and Trade-Offs:

Explains reasoning behind architectural choices.

- HTTPS ensures encrypted, secure communication across all components.
- WebSockets chosen to achieve instant two-way updates instead of polling.
- In-memory aggregation preferred for speed over database aggregation.
- Accepted trade-off: increased memory usage for lower latency and real-time output.

Real-World Considerations:

Places the design in a practical context.

- Best suited for monitoring dashboards or systems requiring immediate feedback.
- Works well with containerized deployments for scalable real-time data pipelines.
- Architecture balances security (via HTTPS) and responsiveness (via WebSockets).

Approach 3: HTTP/2 Streaming + WebSockets (Socket.IO)

Overview:

This approach further improves real-time delivery by introducing HTTP/2 streaming with multiplexed connections while retaining WebSocket support for dashboard updates. It addresses a limitation of Approach 2, where HTTPS WebSocket connections delivered real-time updates but backend scaling and high-frequency metric ingestion could still create bottlenecks. Approach 3 uses HTTP/2 for persistent streams to improve throughput and allow multiple concurrent streams over a single connection, reducing overhead and latency.

Data Flow

These points describe how metrics move from servers to the dashboard and database.

- Server agents send metrics as NDJSON streams via HTTPS POST to `/api/stream`.
- The **proxy** receives requests:
 - `/api/stream` → forwarded to HTTP/2 backend pool (BACKENDS_H2) using persistent HTTP/2 sessions.
 - Other requests (dashboard UI, REST API calls, WebSocket upgrades) → round-robin to standard HTTPS backend pool (BACKENDS_STD).
- Backend HTTP/2 servers parse NDJSON streams and process each metric line.
- Metrics are handled in-memory via `aggMap` for stateful aggregation.
- Aggregated metrics are periodically flushed to MySQL (`MetricRaw` every 2s, `MetricMinuteAgg` every 5s).
- Real-time updates are emitted to the dashboard using WebSockets (Socket.IO).

Architecture Layers

These points summarize the layers and their roles.

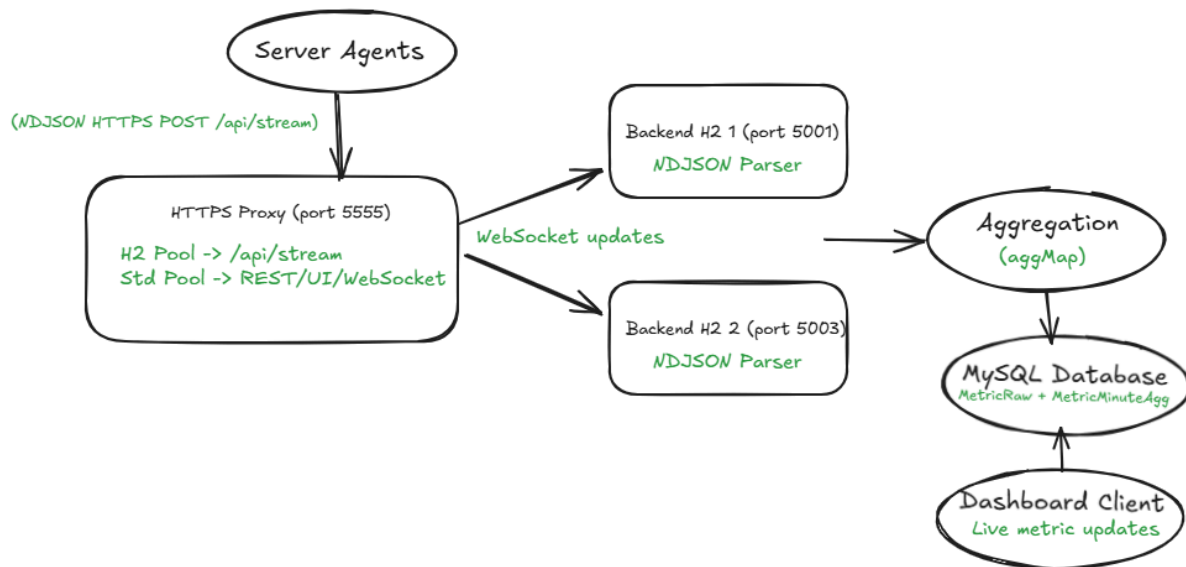
- **Proxy Layer:**
 - HTTPS reverse proxy handling HTTP/2 streams and standard HTTPS/WebSocket traffic.
 - Balances `/api/stream` requests across H2 backend pool using round-robin.
 - Maintains persistent HTTP/2 sessions to backend servers for efficient multiplexed streaming.
 - Handles WebSocket upgrades for live dashboard connections.
- **Backend Layer:**
 - HTTP/2 server for `/api/stream` endpoints.
 - Standard HTTPS server for dashboard and non-stream REST APIs.
 - Parses NDJSON streams and forwards metrics to the aggregator.
- **Aggregation Layer:**
 - In-memory, stateful aggregation per `(server_id, metric_name, minute window)`.
 - Calculates count, sum, min, max, average, and p95 values.
 - Periodically flushes aggregated metrics to MySQL for historical analysis.
- **Storage Layer:** MySQL stores all raw metrics and minute-level aggregates.
- **Dashboard Layer:**
 - Socket.IO frontend receives live metric updates.
 - Dashboard also serves static HTML/JS for visualization.

Core Components

These points highlight the key scripts and their responsibilities.

- `proxy3.js` → Reverse proxy, load balancer, HTTP/2 client sessions, WebSocket upgrade handling.
- `server.js` → Hosts HTTP/2 metric stream server and standard HTTPS server with Socket.IO dashboard.
- `streamAggregator.js` → In-memory stateful aggregator, manages buffers for DB batch inserts, emits real-time updates.
- `stream.routes.js` → Handles `/api/stream` POST ingestion and line-by-line NDJSON processing.
- `dashboard.html` → Connects via Socket.IO to receive live metrics.

Architecture Diagram



OSI Model Breakdown

Shows which OSI layers are active in this design.

- Layer 7: HTTPS + HTTP/2 streaming + WebSockets (Socket.IO)
- Layer 4: TCP
- Layer 3: IP
- Layers 1–2: Network hardware

Real-Time Mechanism

Explains how the system achieves low-latency updates.

- HTTP/2 persistent connections allow multiple streams over a single TCP session.
- NDJSON metric lines are parsed and aggregated in-memory immediately.
- Socket.IO emits real-time updates for live dashboards (<1s latency).
- Batch inserts to MySQL reduce database pressure while maintaining historical data.

Load Distribution

Describes how traffic and processing are balanced.

- `/api/stream` requests are distributed across HTTP/2 backend pool using round-robin.
- Standard HTTPS and WebSocket connections are routed to separate backend pool.
- Proxy maintains persistent HTTP/2 sessions to avoid connection overhead.
- Multiple concurrent streams supported per session (`maxConcurrentStreams`: 1000).

Performance Behavior

Summarizes expected behavior under load.

- High-frequency metric ingestion supported without opening excessive TCP connections.
- In-memory aggregation minimizes latency for live dashboard updates.
- Persistent HTTP/2 streams reduce connection overhead compared to Approach 2.
- MySQL batch inserts prevent write bottlenecks while maintaining historical data.

Testing Observations

- Dashboard receives live updates instantly (<1s).
- NDJSON streams handled without blocking other traffic.
- HTTP/2 connections remain stable under high concurrency.
- Backend memory usage grows with active windows but flush intervals mitigate overload.

When to Use This Approach

- Systems requiring high-throughput real-time streaming.
- Scenarios with hundreds or thousands of servers sending frequent metrics.
- Use when reducing TCP connection overhead and multiplexing streams are important.

Limitations

- Slightly higher backend complexity than Approach 2.
- In-memory aggregation is temporary; restarting backend loses non-persisted metrics.
- Requires valid SSL certificates for HTTPS and HTTP/2.
- Increased memory footprint for in-memory aggregation buffers.

Design Decisions and Trade-Offs

- HTTP/2 chosen for stream multiplexing and reduced overhead.
- In-memory aggregator used to prioritize low latency over strict immediate persistence.
- WebSockets retained for dashboard live updates for backward compatibility.
- Trade-off: slightly more complex architecture for higher performance and scalability.

Real-World Considerations

- Ideal for large-scale monitoring dashboards or IoT telemetry ingestion.
- Works well with containerized or cloud deployments for scaling both backends and streams.
- Balances responsiveness, throughput, and security using HTTPS + HTTP/2 + WebSockets.

System Specifications

- **CPU:** Intel Core i5-10210U @ 1.60GHz (2.11 GHz)
- **RAM:** 8 GB (7.81 GB usable)
- **Storage:** 238 GB SSD + 932 GB HDD
- **Graphics:** NVIDIA GeForce MX110 (2 GB) + Intel UHD Graphics
- **OS:** Windows 10 Pro, 64-bit, Version 22H2

Comparison of Approaches

The table below compares the three approaches, showing protocol, backend type, proxy usage, latency, and failures. P99 values are currently empty and will be updated after further testing.

Approach	Protocol / Mode	Backend Type	Proxy	Avg Latency	Failures	Throughput (req/s)	CPU Usage (%)	Memory (MB)	Winner
1	HTTP/1.1	Stateless	No	3.78s	0%	26.2	30	130	
1	HTTP/1.1	Stateless	Yes	2.66s	0%	36.7	30	130	✓
2	HTTPS + WebSocket	Stateful	No	521ms	0%	189	30–40	200	✓
2	HTTPS + WebSocket	Stateful	Yes	981ms	0%	73	30–40	200	
3	HTTP/2 + WebSocket	Stateful	No	199ms	0%	497	6–7	70	✓
3	HTTP/2 + WebSocket	Stateful	Yes	4.46s	21%	22.2	6–7	70	

Notes from the test results:

- **Approach 3 without proxy** had the best latency and throughput with minimal CPU/memory usage.
- **Proxy generally increased latency** for all approaches, especially noticeable in Approach 3.
- Failures appeared only in **Approach 3 with proxy** (21%).

What I Learned

- Start simple with your code and build gradually. Avoid writing complex code at the beginning; begin with small working parts and grow the project step by step.
- Stream aggregation is faster but increases memory usage, so balance performance and resources.
- Using a proxy or load balancer reduces latency for stateless approaches but can become a bottleneck for high-frequency streaming.
- WebSockets and HTTP/2 make dashboards much more responsive.
- For future improvements: consider fault-tolerant streaming, persistent aggregator storage, and dynamic backend scaling.
- Applied course concepts include request handling patterns, protocol trade-offs, backend execution patterns, and load balancing.

Resources

I downloaded a detailed video explanation of the project for offline reference and included it in the project files: [Real-Time Analytics Dashboard Explanation](#)